

1. Project Overview

Project Description

The project is a dynamic race simulation system designed to visualize competitor rankings in real time using sorting and searching algorithms.

The system allows users to add competitors with their initial speeds and IDs, then start a race simulation where competitors' speeds and distances are updated continuously during each round.

The program automatically reorders competitors according to their current performance to determine:

- First place competitor
- Last place competitor
- Current ranking of all competitors

The project demonstrates the practical use of:

- Sorting Algorithms
- Searching Algorithms
- Queue Data Structure
- Dynamic Simulation

inside a real-time ranking system.

2. Project Objectives

The main objectives of this project are:

- Simulate a dynamic racing environment.
- Track competitors' rankings in real time.
- Apply different sorting algorithms to organize competitors.
- Apply searching algorithms to locate competitors efficiently.
- Compare CPU execution times of algorithms.
- Analyze algorithm efficiency and scalability.
- Demonstrate the importance of algorithm selection in real-world systems.

3. System Features

The system supports the following operations:

1. Add Competitor
2. Show Competitors
3. Start Race Simulation
4. Search Competitor
5. Update Competitor Speed
6. Sort Competitors
7. Exit System

4. OOP Design

Competitor Structure

```
struct Competitor {  
    int id;  
    string name;  
    double speed;  
    double distance;  
};
```

Explanation

Variable	Purpose
id	Unique competitor identifier
name	Competitor name
speed	Current competitor speed
distance	Total traveled distance

5. Data Structures Used

5.1 Vector

```
vector<Competitor> competitors;
```

Why Vector?

Vector is used because:

- Dynamic size
- Fast traversal
- Easy sorting
- Easy searching
- Efficient storage for competitor data

Complexity

Operation	Complexity
Access	$O(1)$
Insertion	$O(1)$ average
Traversal	$O(n)$

5.2 Queue

```
queue<Competitor> raceQueue;
```

Why Queue? Queue is used to simulate competitor starting order using FIFO logic.

Complexity

Operation	Complexity
Push	$O(1)$
Pop	$O(1)$
Front	$O(1)$

6. Program Screens

6.1 Main Menu

```
=====
      Race Visualizer System
=====

1) Add Competitor
2) Show Competitors
3) Start Race
4) Search Competitor
5) Update Speed
6) Sort Competitors
7) Exit
```

Functionality

This screen allows navigation through all system operations using switch-case statements.

6.2 Add Competitor Screen

```
===== Add Competitor =====

Enter Competitor ID:
Enter Competitor Name:
Enter Speed:
```

Functionality

Creates new competitor object
Stores competitor in vector
Pushes competitor into queue

6.3 Show Competitors Screen

ID	Name	Speed	Distance
----	------	-------	----------

Functionality

Displays all competitors using loop traversal.

6.4 Start Race Screen

Functionality

When the race starts:

- Competitor speeds are updated randomly.
- Distances are recalculated.
- Competitors are sorted according to distance.
- Current rankings are displayed.

Speed Update Logic

```
int randomChange = rand() % 21 - 10;
```

This generates random speed changes between: -10 and +10.

Distance Update

```
competitor.distance += competitor.speed;
```

Ali	Omar	Ahmed	Round
60m	80m	100m	1
140m	190m	210m	2
250m	300m	320m	3

The competitors' rankings changed dynamically after speed updates during each race round.

6.5 Search Competitor Screen

Functionality

Allows searching competitors using:

- Competitor ID
- Competitor Name

Algorithms used:

- Linear Search
- Binary Search

6.6 Sort Competitors Screen, Functionality: Allows sorting competitors by: Speed Distance ID

Algorithms used: Quick Sort and Merge Sort

7. Sorting Algorithms

7.1 Quick Sort

Description: Quick Sort is a divide-and-conquer sorting algorithm that selects a pivot element and partitions the array around it. Quick Sort operates efficiently with low memory overhead, making it suitable for dynamic real-time ranking systems.

Time Complexity

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

Advantages

- Very fast in practice
- Efficient for large datasets
- Low memory usage

Disadvantages

- Worst-case complexity can become $O(n^2)$
- Performance depends on pivot selection

Why We Used Quick Sort

Quick Sort was used during dynamic race ranking updates because:

- The ranking changes continuously during the race
- It provides fast average performance
- It handles repeated sorting operations efficiently
- It achieved lower CPU execution time in our tests

CPU Time Result

Average execution time:

0.008363 ms

7.2 Merge Sort

Description: Merge Sort divides the array into smaller parts, sorts them recursively, then merges them.

Time Complexity

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

Advantages

Stable sorting algorithm
Predictable performance
Efficient for large datasets

Disadvantages

Requires additional memory
Slower than Quick Sort in this project

Why We Used Merge Sort

Merge Sort was implemented for comparison purposes because:

- It guarantees $O(n \log n)$ complexity
- It is widely used in large-scale systems
- It helps analyze sorting performance differences

CPU Time Result

Average execution time:

0.0192295 ms

Sorting Algorithms Comparison

Feature	Quick Sort	Merge Sort
Average Complexity	$O(n \log n)$	$O(n \log n)$
Worst Complexity	$O(n^2)$	$O(n \log n)$
Memory Usage	Low	Higher
Stability	Not Stable	Stable
CPU Time in Project	Faster	Slower
Best Usage	Dynamic real-time sorting	Large stable datasets

Final Sorting Conclusion

Quick Sort achieved better performance in this project because the race system continuously updates competitors' rankings in real time. Its lower CPU execution time made it more suitable for dynamic sorting operations.

Merge Sort provided stable and predictable performance, but it required more memory and higher execution time compared to Quick Sort.

Therefore, Quick Sort was selected as the primary sorting algorithm for the race ranking system.

8. Searching Algorithms

Searching Algorithms Comparison

In this project, two searching algorithms were implemented:

- Linear Search

- Binary Search

These algorithms were used to search for competitors using:

- Competitor ID

- Competitor Name

8.1 Linear Search

Description: Linear Search checks elements sequentially until the target competitor is found.

Complexity

Case	Complexity
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$

Advantages

- Simple implementation

- Works on unsorted data

- Easy to maintain

Disadvantages

- Slow for large datasets

- Checks elements sequentially

Why We Used Linear Search

Linear Search was used because:

- Some competitor data may not be sorted

- It is simple for direct sequential searching

- It works immediately without preprocessing

CPU Time Result

Average execution time:

0.00549066 ms

8.2 Binary Search

Description: Binary Search repeatedly divides the sorted data into halves.

Binary Search reduces the search space by half during each iteration, making it significantly faster than Linear Search for sorted datasets.

Complexity

Case	Complexity
Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$

Advantages

- Extremely fast
- Efficient for large datasets
- Reduces search space significantly

Disadvantages

- Requires sorted data
- More complex implementation compared to Linear Search

Why We Used Binary Search

Binary Search was used after sorting competitors because:

- Competitor rankings become sorted after race updates
- Binary Search provides significantly faster lookup performance
- It minimizes CPU execution time

CPU Time Result

Average execution time:

```
0.000244198 ms
```

Searching Algorithms Comparison

Feature	Linear Search	Binary Search
Complexity	$O(n)$	$O(\log n)$
Requires Sorted Data	No	Yes
Speed	Slower	Faster
Implementation	Simple	More Complex
CPU Time in Project	Higher	Lower
Best Usage	Small/unsorted data	Large sorted datasets

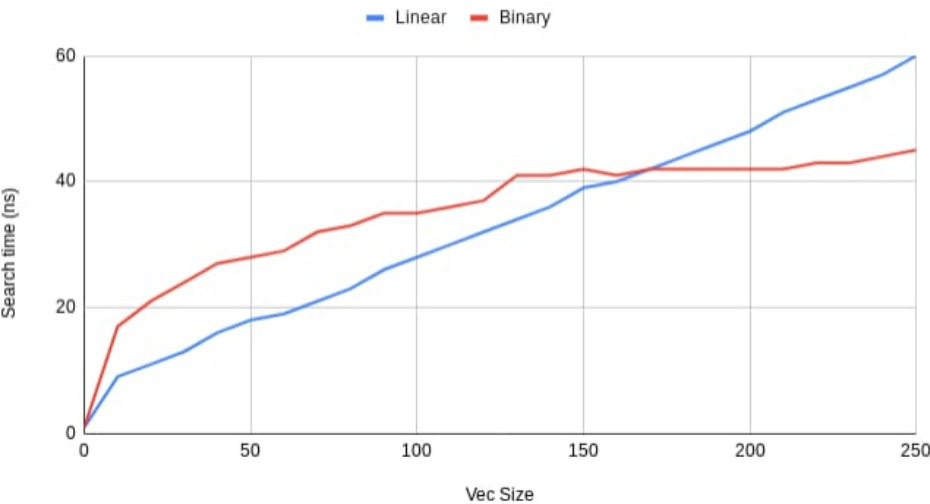
Final Searching Conclusion

Binary Search significantly outperformed Linear Search in this project because competitors were already sorted after ranking updates.

Although Linear Search is easier to implement and works with unsorted data, Binary Search provided much faster execution time and better scalability.

Therefore, Binary Search was selected as the primary searching algorithm after sorting operations were completed.

Linear vs. Binary Search



9. CPU Time Analysis

Why CPU Time Changes: CPU execution time may vary between runs because: Operating system scheduling, Background processes, CPU thread management and Memory state.

Therefore, each algorithm was executed multiple times and the average execution time was calculated. As input sizes increase, algorithm efficiency becomes more important. Efficient sorting and searching algorithms help reduce CPU usage and improve system responsiveness during real-time race updates.

10. Quick Sort CPU Time Results

Run	Time (ms)
1	0.00781250
2	0.00805664
3	0.00781250
4	0.00927734
5	0.00756836
6	0.00903320
7	0.00708008
8	0.00878906
9	0.00781250
Average Time	
0.008363 ms	

11. Merge Sort CPU Time Results

Run	Time (ms)
1	0.01855469
2	0.02075195
3	0.01684570
4	0.02026367
5	0.01806641
6	0.02075195
7	0.01977539
8	0.01928711
9	0.01977539

Average Time

0.0192295 ms

12. Linear Search CPU Time Results

Run	Time (ms)
1	0.005317578
2	0.005580820
3	0.005297852
4	0.005493164
5	0.005346680
6	0.005395508
7	0.005493164
8	0.005297852
9	0.005249023

Average Time

0.00549066 ms

13. Binary Search CPU Time Results

Run	Time (ms)
1	0.00024472
2	0.00024414
3	0.00000000
4	0.00024414
5	0.00000000
6	0.00024414
7	0.00024414
8	0.00000000
9	0.00000000

Average Time

0.000244198 ms

14. Comparative Analysis

Sorting Comparison

Algorithm	Average Time
Quick Sort	0.008363 ms
Merge Sort	0.0192295 ms

Observation

Quick Sort achieved faster execution time than Merge Sort in this project.

Searching Comparison

Algorithm	Average Time
Linear Search	0.00549066 ms
Binary Search	0.000244198 ms

Observation

Binary Search significantly outperformed Linear Search because it reduces the search space by half each iteration.

15. Scalability Discussion

The project supports scalability because:

- Vector handles dynamic competitor storage efficiently.
- Quick Sort and Merge Sort perform efficiently with larger datasets.
- Binary Search provides fast searching performance for sorted data.
- The race simulation logic can support a large number of competitors.

The system can efficiently handle larger numbers of competitors because the implemented algorithms maintain good performance as dataset size increases.

16. Conclusion

This project successfully demonstrated the practical application of sorting algorithms, searching algorithms, and data structures inside a dynamic race simulation environment.

The project showed how algorithm selection directly affects:

- Performance
- CPU execution time
- Scalability
- System efficiency

The implementation proved that Quick Sort and Binary Search provided the best performance among the tested algorithms in this specific scenario.

17. Future Improvements

Possible future improvements include:

- Adding graphical visualization
- Supporting multiplayer races
- Adding more sorting algorithms
- Real-time animation
- Database integration
- Advanced statistics and analytics